

ServiceConnect — Complete Technical Project Documentation

Document Version: 2.4.0

Generation Date: September 19, 2026

Classification: Comprehensive System Architecture & Engineering Reference

Target Audience: Software Engineers, Solutions Architects, QA Leads, DevOps Engineers, and System Administrators

Status: VERIFIED & OPERATIONAL (All 10 Microservices + Vite Frontend passing full End-to-End verification)

1. Executive Summary & Platform Overview

1.1 Mission and Purpose

ServiceConnect is a robust, enterprise-grade, on-demand local services marketplace platform engineered to bridge the gap between consumers seeking dependable home and professional services and certified service providers offering localized trade expertise. The platform provides a seamless digital lifecycle:

1. **Discovery & Exploration:** Unauthenticated consumers can search and discover qualified providers and catalog offerings by category, price, and geographic availability.
2. **Booking & Scheduling:** Authenticated customers can reserve specific service slots matched against the provider's active operating hours, protected by idempotency controls.
3. **Execution & Fulfillment:** Real-time state machine tracking transitioning requests through `PENDING`, `ACCEPTED`, and `COMPLETED` milestones.
4. **Ratings & Feedback:** Verified single-review-per-booking feedback loops recalculating provider aggregate ratings.
5. **Customer Support:** Integrated support ticketing system with priority escalation and multi-party threaded messaging.
6. **Administrative Governance:** Enterprise admin portal providing platform KPIs, provider onboarding approvals/suspensions, global ticket triage, audit logging, and Help Center knowledge base curation.

1.2 System Scope & Scale

- **Backend Architecture:** Microservices architecture comprising **10 distinct Spring Boot 3.x / Java 17+ services**, including Netflix Eureka Service Discovery, Spring Cloud Gateway (reactive Netty), and 8 domain microservices.
- **Data Persistence:** **8 dedicated PostgreSQL databases** adhering strictly to the Database-per-Service pattern, preventing cross-domain database coupling.
- **Frontend Architecture:** Modern Single-Page Application (SPA) built on **React 18.3.1**, **Vite 5.4.6**, **TypeScript 5.5.3**, **Tailwind CSS 3.4.11**, and **shadcn/ui** (Radix UI primitives).
- **Route Surface:** **63 distinct page routes** organized into 5 role-based layouts (`PublicLayout` , `AuthLayout` , `CustomerLayout` , `ProviderLayout` , and `AdminLayout`).
- **Verified Health:** 100% Passing End-to-End automated milestone suite across 18 execution phases with 0 HTTP network errors and 0 unhandled console exceptions.

2. Complete System Architecture

ServiceConnect is designed around high-cohesion, low-coupling microservice principles, with perimeter API Gateway routing, reactive circuit breaking, distributed service discovery, and role-enforced stateless JWT authentication.

2.1 Architecture Diagram

flowchart TD

```

    subgraph Clients["Client Layer"]
        WebBrowser["Web Browser (Chrome / Edge / Safari)"]
        MobileWeb["Mobile Browser / Responsive UI"]
    end

    subgraph Frontend["Frontend Tier (Port 3000)"]
        ViteServer["Vite Dev / Nginx Production Server"]
        ReactApp["React 18 SPA (TypeScript + Tailwind)"]
        ZustandStore["Zustand Auth Store (JWT in LocalStorage)"]
        ReactQuery["TanStack React Query Cache"]
        AxiosClients["Axios Centralized Clients (gatewayClient, userServiceClient, etc.)"]
    end

    subgraph GatewayTier["Perimeter & Edge Tier (Port 8080)"]
        APIGateway["Spring Cloud Gateway (WebFlux / Netty)"]
        RedisCluster["Redis (Rate Limiting Token Bucket)"]
        Resilience["Resilience4j Circuit Breakers"]
    end

    subgraph DiscoveryTier["Registry Tier (Port 8761)"]
        EurekaServer["Netflix Eureka Service Discovery"]
    end

    subgraph DomainServices["Microservices Tier (Ports 8081 - 8088)"]
        AuthService["Auth Service (Port 8081)"]
        • JWT Issuance & Verification
        • BCrypt Hashing
        • Password Reset"]
        UserService["User Service (Port 8082)"]
        • Profiles & User Meta
        • Account Settings"]
        AdminService["Admin Service (Port 8083)"]
        • Platform Metrics
        • Tickets & Messages
        • Help Center & Audit Logs"]
        ProviderService["Provider Service (Port 8084)"]
        • Provider Profiles
        • Operating Hours & Geo
        • Direct Photo Upload"]
        BookingService["Booking Service (Port 8085)"]
        • Service Requests
        • Booking State Machine
        • Availability Validation"]
        CatalogService["Catalog Service (Port 8086)"]
        • Service Offerings
        • Pricing & Durations"]
        ReviewService["Review Service (Port 8087)"]
        • Completed Booking Reviews
        • Rating Aggregation"]
        PaymentService["Payment Service (Port 8088)"]
        • Payment Simulation
        • Transaction Verification"]
    end
  
```

```

subgraph PersistenceTier["Database Tier (PostgreSQL 15+)"]
    DB_Auth[(serviceconnect_auth)]
    DB_User[(serviceconnect_user)]
    DB_Admin[(serviceconnect_admin)]
    DB_Provider[(serviceconnect_provider)]
    DB_Booking[(serviceconnect_booking)]
    DB_Catalog[(serviceconnect_catalog)]
    DB_Review[(serviceconnect_review)]
    DB_Payment[(serviceconnect_payment)]
end

subgraph StorageTier["File Storage Tier"]
    DiskUploads["Local Disk Storage
uploads/providers/{id}/"]
end

%% Wiring connections
WebBrowser -->|HTTP 3000| ViteServer
MobileWeb -->|HTTP 3000| ViteServer
ViteServer --> ReactApp
ReactApp --> ZustandStore
ReactApp --> ReactQuery
ReactApp --> AxiosClients

AxiosClients -->|HTTP /api/v1/**| APIGateway
AxiosClients -->|Vite Proxy /api/users/**| UserService
AxiosClients -->|Vite Proxy /api/payments/**| PaymentService
AxiosClients -->|Direct /api/v1/tickets/**| AdminService

APIGateway --> RedisCluster
APIGateway --> Resilience
APIGateway -->|lb://AUTH-SERVICE| AuthService
APIGateway -->|lb://USER-SERVICE| UserService
APIGateway -->|lb://PROVIDER-SERVICE| ProviderService
APIGateway -->|lb://BOOKING-SERVICE| BookingService
APIGateway -->|lb://CATALOG-SERVICE| CatalogService
APIGateway -->|lb://REVIEW-SERVICE| ReviewService
APIGateway -->|lb://ADMIN-SERVICE| AdminService

DomainServices <-->|Heartbeat & Registration| EurekaServer

AuthService --> DB_Auth
UserService --> DB_User
AdminService --> DB_Admin
ProviderService --> DB_Provider
BookingService --> DB_Booking
CatalogService --> DB_Catalog
ReviewService --> DB_Review
PaymentService --> DB_Payment

ProviderService -->|Multipart Disk I/O| DiskUploads

%% Inter-service HTTP calls
BookingService -->|OpenFeign / RestTemplate| ProviderServiceClient
AdminService -->|OpenFeign / RestTemplate| ProviderServiceClient
ReviewService -->|OpenFeign / RestTemplate| BookingServiceClient

```

2.2 Ingress, Egress & Network Boundaries

- **Perimeter Security:** In production, only the Frontend web server and the API Gateway (Port 8080) are exposed to external traffic via Ingress/Reverse Proxy.
- **Stateless Tokens:** The client presents an HMAC-SHA256 JWT Bearer token in the `Authorization` header. The API Gateway validates tokens, enforces rate limits, and forwards requests to internal microservices.
- **Service Isolation:** Each microservice operates within its private port boundary and manages its own independent schema. No microservice performs direct SQL queries against another service's database.

3. Microservices & Ports Ground Truth Table

The table below reflects the ground truth port configuration, technology stack, and database allocations verified from the active codebase.

Service Name	Port	Spring Boot / Java	Primary Tech Stack	Database Name	Discovery ID	Base Route
service-discovery	8761	3.2.x / Java 17+	Spring Cloud Netflix Eureka	None	SERVICE-DISCOVERY	http://localhost:8761
api-gateway	8080	3.2.x / Java 17+	Spring Cloud Gateway WebFlux, Redis, Resilience4j	None (Redis cache)	API-GATEWAY	http://localhost:8080
auth-service	8081	4.1.x / Java 17+	Spring Boot, Spring Security, Nimbus JWT, Flyway	serviceconnect_auth	AUTH-SERVICE	/api/v1/auth
user-service	8082	3.2.x / Java 17+	Spring Boot, Spring Data JPA, Flyway, Swagger	serviceconnect_user	USER-SERVICE	/api/users
admin-service	8083	3.2.x / Java 17+	Spring Boot, Spring	serviceconnect_admin	ADMIN-SERVICE	/api/v1/admin , /api/v1/tickets

Service Name	Port	Spring Boot / Java	Primary Tech Stack	Database Name	Discovery ID	Base Route	
			Security, Flyway, OpenFeign				
provider-service	8084	3.2.x / Java 17+	Spring Boot, Spring Data JPA, Multipart I/O	serviceconnect_provider	PROVIDER-SERVICE	/api/v1/providers	/
booking-service	8085	3.2.x / Java 17+	Spring Boot, Spring Data JPA, OpenFeign	serviceconnect_booking	BOOKING-SERVICE	/api/v1/bookings	/
catalog-service	8086	3.2.x / Java 17+	Spring Boot, Spring Data JPA, Flyway	serviceconnect_catalog	CATALOG-SERVICE	/api/catalog	/
review-service	8087	3.2.x / Java 17+	Spring Boot, Spring Data JPA, OpenFeign	serviceconnect_review	REVIEW-SERVICE	/api/reviews	/
payment-service	8088	3.2.x / Java 17+	Spring Boot, Spring Data JPA, Flyway	serviceconnect_payment	PAYMENT-SERVICE	/api/payments	/
frontend	3000	Node 20+	React 18.3.1, Vite 5.4.6, TypeScript 5.5.3	LocalStorage / Cache	Client Browser	http://localhost:3000	

4. Frontend Technical Specifications

The ServiceConnect frontend is engineered as a high-performance, accessible Single-Page Application (SPA) designed with a modular design system.

4.1 Technology Dependencies Matrix

- Core Runtime: React ^18.3.1 , React DOM ^18.3.1

- **Build Tool & Bundler:** Vite `^5.4.6` with `@vitejs/plugin-react`
- **Language:** TypeScript `^5.5.3` with strict type checking enabled
- **CSS Architecture:** Tailwind CSS `^3.4.11`, `autoprefixer ^10.4.20`, `postcss ^8.4.45`
- **Component Primitives:** Radix UI (`@radix-ui/react-dialog`, `@radix-ui/react-dropdown-menu`, `@radix-ui/react-accordion`, `@radix-ui/react-select`, `@radix-ui/react-tabs`, `@radix-ui/react-popover`, etc.)
- **Iconography:** `lucide-react ^0.441.0`
- **Client State Management:** `zustand ^5.0.0` with persistent storage middleware
- **Server Data Synchronization:** `@tanstack/react-query ^5.56.0` with dedicated cache management
- **Routing:** `react-router-dom ^6.26.2` with nested layouts and protected route wrappers
- **Form Management & Validation:** `react-hook-form ^7.53.0` coupled with `@hookform/resolvers ^3.9.0` and `zod ^3.23.8`
- **Mapping & Geolocation:** `leaflet ^1.9.4`, `react-leaflet ^4.2.1`, `leaflet.markercluster ^1.5.3`
- **Charts & Telemetry:** `recharts ^2.12.7`
- **Feedback & Notifications:** `sonner ^1.5.0` toast notification system
- **Sanitization & Security:** `dompurify ^3.1.6`

5. Frontend Project Structure & Directory Tree

The frontend repository is structured into distinct functional tiers separating state, networking, UI components, layouts, and pages:

```

cool-bell/
├── public/                # Static assets, logos, favicons
├── src/
│   ├── api/              # API client modules per domain
│   │   ├── admin.ts      # Admin metrics, audit logs, provider approvals
│   │   ├── auth.ts       # Login, register, token refresh, password reset
│   │   ├── booking.ts    # Customer & provider service request workflows
│   │   ├── catalog.ts    # Catalog CRUD, category search
│   │   ├── helpCenter.ts # Public & admin help articles
│   │   ├── payment.ts    # Payment initiation and verification
│   │   ├── provider.ts   # Provider profiles, hours, multipart photo upload
│   │   ├── review.ts     # Customer reviews & ratings
│   │   ├── tickets.ts    # Support tickets & messaging threads
│   │   └── user.ts       # User profiles & account settings
│   ├── components/       # Reusable UI & architectural components
│   │   ├── auth/         # ProtectedRoute, OfflineBanner, RoleGuard
│   │   ├── common/       # Header, Footer, ErrorBoundary, LoadingSpinner
│   │   ├── ui/           # shadcn/ui components (Button, Card, Dialog, Input, etc.)
│   │   └── map/          # Leaflet map widgets and marker clusters
│   ├── hooks/            # Custom React hooks (useDebounce, useAuth, etc.)
│   ├── layouts/          # Structural layout templates
│   │   ├── AdminLayout.tsx # Admin & Support Agent sidebar and top navigation
│   │   ├── AuthLayout.tsx  # Split-screen authentication container
│   │   ├── CustomerLayout.tsx # Customer portal header, navigation, and footer
│   │   ├── ProviderLayout.tsx # Provider management sidebar and navigation
│   │   └── PublicLayout.tsx # Marketing header, main content, and footer
│   ├── lib/              # Library configurations
│   │   ├── axios.ts      # Centralized Axios instances & token refresh interceptor
│   │   └── utils.ts       # Tailwind clsx/twMerge class utility
│   ├── pages/            # Page route components (63 total)
│   │   ├── admin/        # 11 Admin portal views
│   │   ├── customer/     # 22 Customer portal & auth views
│   │   ├── provider/     # 14 Provider portal & auth views
│   │   ├── public/       # 14 Public discovery & legal views
│   │   └── support/      # 2 Support agent views
│   ├── store/            # Client stores
│   │   └── authStore.ts   # Zustand auth state, token storage, user roles
│   ├── types/            # TypeScript interface and enum definitions
│   │   └── index.ts       # Centralized domain types and DTO schemas
│   ├── utils/            # Formatting and date helpers
│   ├── App.tsx           # Route declarations & role guards
│   ├── index.css         # Tailwind directives and custom utility classes
│   └── main.tsx          # Application root mounting TanStack Query & Router
├── .env                  # Environment variable configuration
├── package.json          # Project dependencies and npm scripts
├── tailwind.config.js    # Tailwind theme tokens and color palette
├── tsconfig.json         # TypeScript compiler options
└── vite.config.ts        # Vite server & proxy configuration

```

6. Frontend Complete Route Table

The table below catalogs every registered route in `src/App.tsx`, documenting component mapping, layout wrapper, required role, and functional description.

Route Path	Component File	Layout Wrapper	Role Required	Auth?	Feature Purpose
/	Landing.tsx	PublicLayout	Public	No	Marketplace landing page with search and category filters
/services	Services.tsx	PublicLayout	Public	No	Public marketplace listing with search and category filters
/providers	ProviderDiscovery.tsx	PublicLayout	Public	No	Public provider discovery with direct geolocation map and filters
/providers/:id	ProviderPublicProfile.tsx	PublicLayout	Public	No	Public provider profile with operating hours and ratings
/help	HelpCenter.tsx	PublicLayout	Public	No	Help center knowledge base and article browser
/help/category/:category	HelpCategory.tsx	PublicLayout	Public	No	Help center category filter
/help/article/:slug	HelpArticle.tsx	PublicLayout	Public	No	Full article content view with help center rating
/privacy	LegalPage.tsx	PublicLayout	Public	No	Privacy disclosure
/terms	LegalPage.tsx	PublicLayout	Public	No	Terms of service, customer support, and content terms
/cookies	LegalPage.tsx	PublicLayout	Public	No	Cookie declaration

Route Path	Component File	Layout Wrapper	Role Required	Auth?	Feat Purp
/refunds	LegalPage.tsx	PublicLayout	Public	No	Cust refur polic guid
/cancellation	LegalPage.tsx	PublicLayout	Public	No	Book canc term time
/shipping	LegalPage.tsx	PublicLayout	Public	No	Phys mate deliv polic
/returns	LegalPage.tsx	PublicLayout	Public	No	Retu for s equi
/disclaimer	LegalPage.tsx	PublicLayout	Public	No	Platf liabil discl
/accessibility	LegalPage.tsx	PublicLayout	Public	No	Acce com state
/dpa	LegalPage.tsx	PublicLayout	Public	No	Data Proc Add
/acceptable-use	LegalPage.tsx	PublicLayout	Public	No	Platf acce use r
/security	LegalPage.tsx	PublicLayout	Public	No	Syste secu mea: discl polic
/responsible-disclosure	LegalPage.tsx	PublicLayout	Public	No	Vuln repo proc
/community-guidelines	LegalPage.tsx	PublicLayout	Public	No	Com inter code conc
/cookie-preferences	CookiePreferences.tsx	PublicLayout	Public	No	Inter cook cate cons pref

Route Path	Component File	Layout Wrapper	Role Required	Auth?	Feat Purp
/404	NotFound.tsx	PublicLayout	Public	No	404 Four page
/403	Forbidden.tsx	PublicLayout	Public	No	403 Forb role page
/500	ServerError.tsx	PublicLayout	Public	No	500 Error notif page
/maintenance	Maintenance.tsx	PublicLayout	Public	No	Sche syste main scree
/session-expired	SessionExpired.tsx	PublicLayout	Public	No	JWT expir auth notic
/dashboard	DashboardRedirect	N/A	Authenticated	Yes	Smal base redir
/customer/login	CustomerLogin.tsx	AuthLayout	Public	No	Cust emai pass auth
/customer/register	CustomerRegister.tsx	AuthLayout	Public	No	Cust acco regis
/customer/verify-email	CustomerVerifyEmail.tsx	AuthLayout	Public	No	Cust emai verifi chall
/customer/forgot-password	CustomerForgotPassword.tsx	AuthLayout	Public	No	Cust pass reset form
/customer/reset-password	CustomerResetPassword.tsx	AuthLayout	Public	No	Verif code pass subn
/customer/dashboard	CustomerDashboard.tsx	CustomerLayout	CUSTOMER	Yes	Cust hom with

Route Path	Component File	Layout Wrapper	Role Required	Auth?	Feat Purp
					book stats
/customer/profile	CustomerProfile.tsx	CustomerLayout	CUSTOMER	Yes	Cust persi profi man.
/customer/settings	CustomerSettings.tsx	CustomerLayout	CUSTOMER	Yes	Cust acco prefe and notif
/customer/security	CustomerSecurity.tsx	CustomerLayout	CUSTOMER	Yes	Pass chan sessi term
/customer/services	CustomerServices.tsx	CustomerLayout	CUSTOMER	Yes	Auth servi catal disco
/customer/providers	CustomerProviders.tsx	CustomerLayout	CUSTOMER	Yes	Auth prov direc
/customer/providers/:id	CustomerProviderDetail.tsx	CustomerLayout	CUSTOMER	Yes	Prov deta book requ trigg
/customer/bookings	CustomerBookings.tsx	CustomerLayout	CUSTOMER	Yes	Cust book histc statu
/customer/bookings/:id	CustomerBookingDetail.tsx	CustomerLayout	CUSTOMER	Yes	Deta book prov actic
/customer/payment/:bookingId	CustomerPayment.tsx	CustomerLayout	CUSTOMER	Yes	Simu card chec form
/customer/payment/success	PaymentSuccess.tsx	CustomerLayout	CUSTOMER	Yes	Paym com conf page
/customer/payment/failed	PaymentFailed.tsx	CustomerLayout	CUSTOMER	Yes	Paym failur

Route Path	Component File	Layout Wrapper	Role Required	Auth?	Feat Purp
					with trigg
/customer/payment/pending	PaymentPending.tsx	CustomerLayout	CUSTOMER	Yes	Asyn payn verifi state
/customer/reviews	CustomerReviews.tsx	CustomerLayout	CUSTOMER	Yes	Revie auth com book
/customer/support	CustomerSupport.tsx	CustomerLayout	CUSTOMER	Yes	Supp creat
/customer/support/tickets	CustomerTickets.tsx	CustomerLayout	CUSTOMER	Yes	List c custo subn supp ticke
/customer/support/tickets/:id	CustomerTicketDetail.tsx	CustomerLayout	CUSTOMER	Yes	Ticke mess threa supp
/provider/login	ProviderLogin.tsx	AuthLayout	Public	No	Prov auth
/provider/register	ProviderRegister.tsx	AuthLayout	Public	No	Prov busin regis
/provider/verify-email	ProviderVerifyEmail.tsx	AuthLayout	Public	No	Prov verifi
/provider/forgot-password	ProviderForgotPassword.tsx	AuthLayout	Public	No	Prov pass reset
/provider/reset-password	ProviderResetPassword.tsx	AuthLayout	Public	No	Prov pass reset com
/provider/onboarding	ProviderOnboarding.tsx	ProviderLayout	PROVIDER	Yes	Prov profi busin onbo
/provider/dashboard	ProviderDashboard.tsx	ProviderLayout	PROVIDER	Yes	Prov dash and requ

Route Path	Component File	Layout Wrapper	Role Required	Auth?	Feat Purp
/provider/profile	ProviderProfile.tsx	ProviderLayout	PROVIDER	Yes	Prov busin profi bio €
/provider/catalog	ProviderCatalog.tsx	ProviderLayout	PROVIDER	Yes	Prov catal creat prici
/provider/availability	ProviderAvailability.tsx	ProviderLayout	PROVIDER	Yes	Wee oper hour sche
/provider/bookings	ProviderBookings.tsx	ProviderLayout	PROVIDER	Yes	Incoi book requ man.
/provider/bookings/:id	ProviderBookingDetail.tsx	ProviderLayout	PROVIDER	Yes	Acce com book cont
/provider/photos	ProviderPhotos.tsx	ProviderLayout	PROVIDER	Yes	Mult direc uplo galle
/provider/settings	ProviderSettings.tsx	ProviderLayout	PROVIDER	Yes	Prov acco setti pass
/admin/login	AdminLogin.tsx	AuthLayout	Public	No	Adm Supp Ager
/admin/forgot-password	AdminForgotPassword.tsx	AuthLayout	Public	No	Adm pass reset
/admin/reset-password	AdminResetPassword.tsx	AuthLayout	Public	No	Adm pass reset com
/admin/dashboard	AdminDashboard.tsx	AdminLayout	ADMIN	Yes	Platf over oper healt
/admin/providers	AdminProviders.tsx	AdminLayout	ADMIN	Yes	Prov appr

Route Path	Component File	Layout Wrapper	Role Required	Auth?	Feat Purp
					statu gove
/admin/providers/:id	AdminProviderDetail.tsx	AdminLayout	ADMIN	Yes	Prov inspe appr susp
/admin/tickets	AdminTickets.tsx	AdminLayout	ADMIN	Yes	Glob supp triag assig
/admin/tickets/:id	AdminTicketDetail.tsx	AdminLayout	ADMIN	Yes	Adm reply statu man
/admin/help-center	AdminHelpCenter.tsx	AdminLayout	ADMIN	Yes	Help articl repo and
/admin/help-center/new	AdminHelpArticleForm.tsx	AdminLayout	ADMIN	Yes	Help auth form
/admin/help-center/:id	AdminHelpArticleForm.tsx	AdminLayout	ADMIN	Yes	Help editi publ
/admin/audit-logs	AdminAuditLogs.tsx	AdminLayout	ADMIN	Yes	Syste secu adm audi
/support/tickets	SupportTickets.tsx	AdminLayout	SUPPORT_AGENT	Yes	Assig supp que
/support/tickets/:id	SupportTicketDetail.tsx	AdminLayout	SUPPORT_AGENT	Yes	Supp mess and resol

7. Frontend API Integration Architecture

The client application integrates with backend services through specialized Axios client instances defined in `src/lib/axios.ts` configured with token injection, automatic refresh synchronization, and multipart header handling.

7.1 Centralized Axios Clients

1. `gatewayClient` (`baseURL: ''`): The primary client proxying requests through the Spring Cloud Gateway. Handles authentication, booking workflows, catalog items, reviews, and Help Center public search.
2. `userServiceClient` (`baseURL: ''`): Routes directly to the User Service via the Vite development proxy (`/api/users/**`) to prevent CORS preflight blocking on direct port `8082` .
3. `providerClient` (`baseURL: ''`): Routes provider discovery and profile endpoints via Vite proxy (`/api/v1/providers/**`) to port `8084` .
4. `adminServiceClient` (`baseURL: VITE_ADMIN_SERVICE_URL ?? 'http://localhost:8083'`): Communicates with the Admin Service for ticketing, support messaging, and audit logs.
5. `paymentClient` (`baseURL: ''`): Routes payment operations through Vite proxy (`/api/payments/**`) to port `8088` .

7.2 Request Interceptor: JWT Injection & Multipart Boundaries

Every outgoing request executes the request interceptor:

- Inspects `useAuthStore.getState().accessToken` .
- If an access token exists, attaches `Authorization: Bearer <token>` .
- **Multipart Boundary Preservation:** When `config.data instanceof FormData` , the interceptor executes `delete config.headers['Content-Type']` . This ensures the browser automatically generates the dynamic boundary header (e.g. `multipart/form-data; boundary=----WebKitFormBoundary...`), preventing server-side multipart parsing failures.

7.3 Response Interceptor: Seamless Token Refresh Queue

When an API responds with `HTTP 401 Unauthorized` :

1. If the failed request is already marked with `_retry: true` , the error is immediately rejected.
2. If no refresh token exists, authenticated sessions trigger `onRefreshFailure()` which clears client auth and redirects to `/session-expired` .
3. If a refresh operation is currently in-flight (`isRefreshing === true`), incoming requests are enqueued into `refreshSubscribers` awaiting the new token.
4. If this is the initial 401:
 - Sets `isRefreshing = true` and `originalRequest._retry = true` .

- Executes `POST /api/v1/auth/refresh` with `{ refreshToken }`.
- Updates Zustand state with new `accessToken` and `refreshToken`.
- Resolves all enqueued subscriber requests and retries the original request.
- Resets `isRefreshing = false`.

7.4 Vite Development Proxy Configuration (`vite.config.ts`)

```
server: {
  port: 3000,
  proxy: {
    '/api/v1/catalog': {
      target: 'http://localhost:8086',
      changeOrigin: true,
      rewrite: (path) => path.replace(/^\/api\/v1\/catalog/, '/api/catalog'),
    },
    '/api/v1/providers': {
      target: 'http://localhost:8084',
      changeOrigin: true,
    },
    '/api/users': {
      target: 'http://localhost:8082',
      changeOrigin: true,
    },
    '/api/payments': {
      target: 'http://localhost:8088',
      changeOrigin: true,
    },
    '/api/v1': {
      target: 'http://localhost:8080',
      changeOrigin: true,
    },
  },
}
```

8. Backend Microservices Overview

Each microservice encapsulates a cohesive business domain, exposing versioned REST APIs, executing Flyway database migrations, and validating requests using Jakarta Bean Validation.

8.1 Service Inventory & Specifications

1. Service Discovery (`service-discovery`)

- **Port:** 8761
- **Stack:** Spring Cloud Netflix Eureka Server

- **Role:** Maintains the dynamic service registry of all live microservice instances. Microservices send heartbeats every 10 seconds; leases expire after 30 seconds.

2. API Gateway (`api-gateway`)

- **Port:** `8080`
- **Stack:** Spring Cloud Gateway (WebFlux / Netty), Spring Data Redis, Resilience4j
- **Role:** Perimeter reverse proxy, Redis-backed rate limiting per IP, circuit breaker fallback execution, and path rewriting (`/api/v1/reviews/**` → `/api/reviews/**`).

3. Auth Service (`auth-service`)

- **Port:** `8081`
- **Stack:** Spring Boot 4.1.x / 3.2.x, Spring Security, Nimbus JOSE JWT, PostgreSQL
- **Database:** `serviceconnect_auth`
- **Role:** User authentication, password encryption (BCrypt strength 12), JWT generation, refresh token rotation, verification code generation (email/phone), and account deletion.

4. User Service (`user-service`)

- **Port:** `8082`
- **Stack:** Spring Boot 3.2.x, Spring Data JPA, PostgreSQL, Springdoc OpenAPI
- **Database:** `serviceconnect_user`
- **Role:** User personal profiles (first name, last name, phone, address, avatar), onboarding status tracking, and account communication settings.

5. Admin Service (`admin-service`)

- **Port:** `8083`
- **Stack:** Spring Boot 3.2.x, Spring Security, Flyway, PostgreSQL, OpenFeign
- **Database:** `serviceconnect_admin`
- **Role:** Platform KPI aggregation, customer support tickets, support agent messaging, provider status governance, Help Center article CRUD, and security audit logging.

6. Provider Service (`provider-service`)

- **Port:** `8084`
- **Stack:** Spring Boot 3.2.x, Spring Data JPA, PostgreSQL, Multipart File I/O
- **Database:** `serviceconnect_provider`

- **Role:** Provider business profiles, geolocation coordinates, weekly operating availability, direct multipart portfolio photo upload, and public image streaming.

7. Booking Service (`booking-service`)

- **Port:** `8085`
- **Stack:** Spring Boot 3.2.x, Spring Data JPA, OpenFeign, PostgreSQL
- **Database:** `serviceconnect_booking`
- **Role:** Service request lifecycle, slot scheduling validation, idempotency key enforcement, and state transitions (`PENDING` , `ACCEPTED` , `REJECTED` , `COMPLETED` , `CANCELLED`).

8. Catalog Service (`catalog-service`)

- **Port:** `8086`
- **Stack:** Spring Boot 3.2.x, Spring Data JPA, PostgreSQL
- **Database:** `serviceconnect_catalog`
- **Role:** Service offerings catalog, pricing models, estimated durations, category categorization, and provider service activation/deactivation.

9. Review Service (`review-service`)

- **Port:** `8087`
- **Stack:** Spring Boot 3.2.x, Spring Data JPA, OpenFeign, PostgreSQL
- **Database:** `serviceconnect_review`
- **Role:** Customer review submission, 1-to-5 star ratings, single-review-per-booking validation, and provider aggregate score recalculation.

10. Payment Service (`payment-service`)

- **Port:** `8088`
 - **Stack:** Spring Boot 3.2.x, Spring Data JPA, Flyway, PostgreSQL
 - **Database:** `serviceconnect_payment`
 - **Role:** Payment simulation, transaction record storage, and payment verification. Note: Uses UUID identifiers mapped via frontend adapter.
-

9. Complete API Endpoint Reference

The following catalog provides the exact endpoint contract derived directly from backend controller source code across all services.

9.1 Auth Service Endpoints (AuthController.java — Port 8081)

Verb	Path	Auth / Role	Request Body	Response Body
POST	/api/v1/auth/register	Public	{ email, password, firstName, lastName, phoneNumber }	{ userId, email, role, accessToken, refreshToken }
POST	/api/v1/auth/register/provider	Public	{ email, password, firstName, lastName, phoneNumber, businessName }	{ userId, email, role, accessToken, refreshToken }
POST	/api/v1/auth/login	Public	{ email, password }	{ userId, email, role, accessToken, refreshToken }
GET	/api/v1/auth/me	Authenticated	None	{ id, email, role }
POST	/api/v1/auth/refresh	Public	{ refreshToken }	{ accessToken, refreshToken }
POST	/api/v1/auth/logout	Public	{ refreshToken }	None
POST	/api/v1/auth/logout-all	Authenticated	None	None
POST	/api/v1/auth/verification/email/request	Public	{ verificationToken }	None
POST	/api/v1/auth/verification/email/verify	Public	{ verificationToken, code }	None
POST	/api/v1/auth/verification/phone/request	Public	{ verificationToken }	None
POST	/api/v1/auth/verification/phone/verify	Public	{ verificationToken, code }	None
POST	/api/v1/auth/password/forgot	Public	{ email }	{ message }
POST	/api/v1/auth/password/reset	Public	{ email, code, newPassword }	None

Verb	Path	Auth / Role	Request Body	Response Body
POST	/api/v1/auth/password/change	Authenticated	{ currentPassword, newPassword }	None
GET	/api/v1/auth/me/security	Authenticated	None	{ twoFactorEnabled, activeSessionsCount, lastPasswordChange }
DELETE	/api/v1/auth/me	Authenticated	{ password }	None
GET	/api/v1/auth/internal/users/{userId}/role	ROLE_ADMIN	None	{ userId, role }

9.2 User Service Endpoints (UserController.java — Port 8082)

Verb	Path	Auth / Role	Request Body	Response Body	HT Sta
POST	/api/users	Authenticated	{ firstName, lastName, phoneNumber, avatarUrl, address, city, state, postalCode, bio }	UserResponse object	201 Creat
GET	/api/users	Authenticated	None	List<UserResponse>	200
GET	/api/users/me	Authenticated	None	UserResponse object	200
GET	/api/users/me/onboarding	Authenticated	None	{ completed, stepsCompleted }	200
PUT	/api/users/me	Authenticated	UserRequest	UserResponse	200
GET	/api/users/me/settings	Authenticated	None	AccountSettingsResponse	200
PUT	/api/users/me/settings	Authenticated	AccountSettingsRequest	AccountSettingsResponse	200
GET	/api/users/{id}	Authenticated	None	UserResponse	200
PUT	/api/users/{id}	Authenticated	UserRequest	UserResponse	200
DELETE	/api/users/{id}	Authenticated	None	None	204 Cont
GET	/api/users/{id}/phone	Internal	None	Raw string phone number	200

9.3 Provider Service Endpoints (provider-service — Port 8084)

Provider Profiles (ProviderController.java)

Verb	Path	Auth / Role	Parameters / Body	Response Body
POST	/api/v1/providers	ROLE_PROVIDER	Body: CreateProviderRequest	ProviderResponse
GET	/api/v1/providers	Public	Query: page , size	PageResponse<Proc
GET	/api/v1/providers/{providerId}/public	Public	Path: providerId	ProviderResponse
GET	/api/v1/providers/me	ROLE_PROVIDER	None	ProviderResponse
GET	/api/v1/providers/{providerId}	Provider/Admin	Path: providerId	ProviderResponse
GET	/api/v1/providers/user/{userId}	Provider/Admin	Path: userId	ProviderResponse
PUT	/api/v1/providers/{providerId}	ROLE_PROVIDER	Body: CreateProviderRequest	ProviderResponse
PATCH	/api/v1/providers/{providerId}/location	ROLE_PROVIDER	Body: { latitude, longitude }	ProviderResponse
GET	/api/v1/providers/onboarding/status	ROLE_PROVIDER	None	ProviderOnboardi
GET	/api/v1/providers/admin/all	ROLE_ADMIN	Query: status , page , size	PageResponse<Proc
PATCH	/api/v1/providers/{providerId}/status	ROLE_ADMIN	Body: { status: "APPROVED"	"SUSPENDED" }
DELETE	/api/v1/providers/{providerId}	ROLE_ADMIN	Path: providerId	None

Operating Hours (ProviderAvailabilityController.java)

Verb	Path	Auth / Role	Parameters / Body	
POST	/api/v1/providers/{providerId}/availability	ROLE_PROVIDER	Body: { dayOfWeek, startTime, endTime }	
GET	/api/v1/providers/{providerId}/availability	ROLE_PROVIDER	Path: providerId	
GET	/api/v1/providers/{providerId}/availability/active	Public	Path: providerId	
PUT	/api/v1/providers/{providerId}/availability/{id}	ROLE_PROVIDER	Body: ProviderAvailabilityRequest	
DELETE	/api/v1/providers/{providerId}/availability/{id}	ROLE_PROVIDER	Path: providerId , id	

Direct Multipart Photos (ProviderPhotoController.java & ProviderPhotoFilesController.java)

Verb	Path	Auth / Role	Parameters / Body	Response
POST	/api/v1/providers/{providerId}/photos/upload	Provider/Admin	Multipart: file , displayOrder	ProviderPhoto
GET	/api/v1/providers/{providerId}/photos/files/{filename}	Public	Path: providerId , filename	Binary Image (image/png image/webp
GET	/api/v1/providers/photos/files/{filename}	Public	Path: filename	Binary Image
GET	/api/v1/providers/{providerId}/photos	Public	Path: providerId	List<ProviderPhoto>
DELETE	/api/v1/providers/{providerId}/photos/{photoId}	Provider/Admin	Path: providerId , photoId	None

9.4 Booking Service Endpoints (BookingController.java — Port 8085)

Verb	Path	Auth / Role	Headers / Body	Response Body
POST	/api/v1/bookings/requests	ROLE_CUSTOMER	Header: x-Idempotency-Key Body: { providerId, catalogItemId, scheduledStartTime, scheduledEndTime, notes }	ServiceRequest
GET	/api/v1/bookings/requests/{requestId}	Cust/Prov/Admin	Path: requestId	ServiceRequest
GET	/api/v1/bookings/customers/requests	ROLE_CUSTOMER	Query: page , size	PageResponse
PATCH	/api/v1/bookings/requests/{requestId}/cancel	ROLE_CUSTOMER	Path: requestId	ServiceRequest
GET	/api/v1/bookings/providers/requests	ROLE_PROVIDER	Query: page , size	PageResponse
GET	/api/v1/bookings/providers/requests/status	ROLE_PROVIDER	Query: status , page , size	PageResponse
PATCH	/api/v1/bookings/requests/{requestId}/status	ROLE_PROVIDER	Body: { status: "ACCEPTED"	"REJECTED"

9.5 Catalog Service Endpoints (CatalogItemController.java — Port 8086)

Verb	Path	Auth / Role	Parameters / Body	Response Body
POST	/api/catalog	ROLE_PROVIDER	Body: { title, description, category, price, durationMinutes }	CatalogItemResponse
GET	/api/catalog/{id}	Public	Path: id	CatalogItemResponse
GET	/api/catalog	Public	Query: search , category , page , size	PageResponse<CatalogItem>
GET	/api/catalog/provider/{providerId}	Public	Path: providerId , Query: page , size	PageResponse<CatalogItem>
PUT	/api/catalog/{id}	ROLE_PROVIDER	Body: CatalogItemRequest	CatalogItemResponse
PATCH	/api/catalog/{id}/activate	ROLE_PROVIDER	Path: id	CatalogItemResponse
DELETE	/api/catalog/{id}	ROLE_PROVIDER	Path: id	None

9.6 Review Service Endpoints (ReviewController.java — Port 8087)

Verb	Path	Auth / Role	Parameters / Body	Response Body
POST	/api/reviews	ROLE_CUSTOMER	Body: { bookingId, providerId, rating, comment }	ReviewResponse
GET	/api/reviews/{id}	Authenticated	Path: id	ReviewResponse
GET	/api/reviews/booking/{bookingId}	Authenticated	Path: bookingId	ReviewResponse
GET	/api/reviews/provider/{providerId}	Public	Path: providerId , Query: page , size	PageResponse<ReviewResponse>
GET	/api/reviews/customer/{customerId}	Authenticated	Path: customerId , Query: page , size	PageResponse<ReviewResponse>

Verb	Path	Auth / Role	Parameters / Body	Response Body
GET	/api/reviews	Authenticated	Query: page , size	PageResponse<ReviewResponse>
PUT	/api/reviews/{id}	ROLE_CUSTOMER	Body: ReviewRequest	ReviewResponse
DELETE	/api/reviews/{id}	ROLE_CUSTOMER	Path: id	None

9.7 Payment Service Endpoints (PaymentController.java — Port 8088)

Verb	Path	Auth / Role	Request Body	Response Body	HTTP Status
POST	/api/payments	Authenticated	{ bookingId, userId, amount, currency, paymentMethod }	CreatePaymentResponse	200 OK
POST	/api/payments/verify	Authenticated	{ paymentId, transactionRef, status }	VerifyPaymentResponse	200 OK

9.8 Admin & Support Service Endpoints (admin-service — Port 8083)

Support Tickets (TicketController.java)

Verb	Path	Auth / Role	Parameters / Body	Response B
POST	/api/v1/tickets	ROLE_CUSTOMER	Body: { title, description, category, priority }	TicketResp
GET	/api/v1/tickets	ROLE_CUSTOMER	Query: page , size	PageRespon
GET	/api/v1/tickets/{ticketId}	ROLE_CUSTOMER	Path: ticketId	TicketResp
GET	/api/v1/tickets/assigned	ROLE_SUPPORT_AGENT	Query: page , size	PageRespon
GET	/api/v1/tickets/{ticketId}/assigned	ROLE_SUPPORT_AGENT	Path: ticketId	TicketResp
PATCH	/api/v1/tickets/{ticketId}/assigned	ROLE_SUPPORT_AGENT	Body: { status, priority }	TicketResp
GET	/api/v1/tickets/admin	ROLE_ADMIN	Query: page , size	PageRespon
GET	/api/v1/tickets/admin/status/{status}	ROLE_ADMIN	Path: status , Query: page , size	PageRespon

Verb	Path	Auth / Role	Parameters / Body	Response B
GET	/api/v1/tickets/admin/{ticketId}	ROLE_ADMIN	Path: ticketId	TicketResp
PATCH	/api/v1/tickets/admin/{ticketId}	ROLE_ADMIN	Body: UpdateTicketRequest	TicketResp
PATCH	/api/v1/tickets/admin/{ticketId}/assignment	ROLE_ADMIN	Body: { agentId }	TicketResp

Threaded Messages (SupportMessageController.java)

Verb	Path	Auth / Role	Parameters / Body	Response Body
POST	/api/v1/tickets/{ticketId}/messages	ROLE_CUSTOMER	Body: { content }	SupportMessageResponse
GET	/api/v1/tickets/{ticketId}/messages	ROLE_CUSTOMER	Path: ticketId , Query: page , size	PageResponse<SupportMes
POST	/api/v1/tickets/{ticketId}/agent/messages	ROLE_SUPPORT_AGENT	Body: { content }	SupportMessageResponse
GET	/api/v1/tickets/{ticketId}/agent/messages	ROLE_SUPPORT_AGENT	Path: ticketId , Query: page , size	PageResponse<SupportMes
POST	/api/v1/tickets/{ticketId}/admin/messages	ROLE_ADMIN	Body: { content }	SupportMessageResponse
GET	/api/v1/tickets/{ticketId}/admin/messages	ROLE_ADMIN	Path: ticketId , Query: page , size	PageResponse<SupportMes

Help Center Articles (HelpCenterController.java & AdminHelpCenterController.java)

Verb	Path	Auth / Role	Parameters / Body	Response Body
GET	/api/v1/help-center/categories	Public	None	List<String> (e.g. ["BOOKINGS", "PAYMENTS"])
GET	/api/v1/help-center/articles	Public	Query: category , search , page , size	PageResponse<HelpArticleResponse>

Verb	Path	Auth / Role	Parameters / Body	Response Body
GET	/api/v1/help-center/articles/{slug}	Public	Path: slug	HelpArticleResponse
POST	/api/v1/admin/help-center/articles	ROLE_ADMIN	Body: { title, slug, content, category, displayOrder, published }	HelpArticleResponse
GET	/api/v1/admin/help-center/articles	ROLE_ADMIN	Query: page , size	PageResponse<HelpArticleResponse>
GET	/api/v1/admin/help-center/articles/{id}	ROLE_ADMIN	Path: id	HelpArticleResponse
PUT	/api/v1/admin/help-center/articles/{id}	ROLE_ADMIN	Body: UpdateHelpArticleRequest	HelpArticleResponse
DELETE	/api/v1/admin/help-center/articles/{id}	ROLE_ADMIN	Path: id	None

Security Audit Logs (AuditLogController.java)

Verb	Path	Auth / Role	Parameters	Response Body	HTTP Status
GET	/api/v1/admin/audit-logs	ROLE_ADMIN	Query: page , size , sort	PageResponse<AuditLogResponse>	200 OK
GET	/api/v1/admin/audit-logs/actor/{actorId}	ROLE_ADMIN	Path: actorId	PageResponse<AuditLogResponse>	200 OK
GET	/api/v1/admin/audit-logs/role/{role}	ROLE_ADMIN	Path: role	PageResponse<AuditLogResponse>	200 OK
GET	/api/v1/admin/audit-logs/action/{action}	ROLE_ADMIN	Path: action	PageResponse<AuditLogResponse>	200 OK

10. Authentication & Authorization Architecture

Authentication across ServiceConnect is completely stateless, relying on JSON Web Tokens (JWT) signed via HMAC-SHA256 with role-based claims.

10.1 Token Lifecycle & Payload Schema

- **Signing Algorithm:** HMAC-SHA256 (HS256) utilizing a cryptographically secure 256-bit secret (\${JWT_SECRET}).
- **Access Token Expiration:** 15 minutes (900 seconds).
- **Access Token Claims:**

- `sub` (Subject): String representation of the numeric User ID (e.g. `"22"`).
- `email` : Authenticated user email (e.g. `"customer@serviceconnect.com"`).
- `role` : User role string without prefix (e.g. `"CUSTOMER"` , `"PROVIDER"` , `"SUPPORT_AGENT"` , `"ADMIN"`).
- `iss` : Token issuer string (`serviceconnect-auth`).
- `iat` : Issued-at Unix epoch timestamp.
- `exp` : Expiration Unix epoch timestamp.
- **Refresh Token Lifecycle:**
 - Opaque cryptographically random token string stored in the PostgreSQL `refresh_tokens` table as a SHA-256 hash.
 - Expiration duration: 7 days (`604,800` seconds).
 - Single-use rotation: When `POST /api/v1/auth/refresh` is called, the old refresh token is marked `revoked = true` and a new access/refresh pair is issued.

10.2 Spring Security Filter Chain

Each downstream microservice configures a stateless Spring Security filter chain:

1. **JwtAuthenticationFilter / Nimbus Decoder:** Extracts the Bearer token from `Authorization: Bearer <token>` .
2. **Signature & Expiry Check:** Validates the cryptographic signature against the shared secret and checks timestamp boundaries.
3. **Granted Authority Mapping:** Maps the `role` claim into a Spring Security authority with prefix `ROLE_` (e.g. claim `"ADMIN"` becomes `SimpleGrantedAuthority("ROLE_ADMIN")`).
4. **SecurityContext Injection:** Injects the authenticated principal into `SecurityContextHolder.getContext().getAuthentication()` .
5. **Method Security:** Controllers declare `@PreAuthorize("hasRole('...')")` or `@PreAuthorize("hasAnyRole('...')")` to enforce endpoint-level security before controller methods execute.

11. Role-Based Access Control (RBAC) Matrix

The table below documents the strict permissions assigned to every actor across the application's functional surface.

Functional Area / Route / Endpoint	Public / Guest	CUSTOMER	PROVIDER	SUPPORT_AGENT	ADMIN
Public Landing & Legal Pages	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed
Catalog Exploration (/services)	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed
Provider Discovery (/providers)	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed
Help Center Search & Read	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed
Provider Portfolio Photo Viewing	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed	✓ Allowed
Customer Dashboard & Bookings	✗ 401	✓ Full Access	✗ 403	✗ 403	✗ 403
Service Request Creation	✗ 401	✓ Full Access	✗ 403	✗ 403	✗ 403
Booking Payment Simulation	✗ 401	✓ Full Access	✗ 403	✗ 403	✗ 403
Post-Service Review Submission	✗ 401	✓ Full Access	✗ 403	✗ 403	✗ 403
Support Ticket Creation (Customer)	✗ 401	✓ Full Access	✗ 403	✗ 403	✗ 403
Provider Onboarding & Profile Edit	✗ 401	✗ 403	✓ Full Access	✗ 403	✗ 403
Catalog Service Creation/Edit	✗ 401	✗ 403	✓ Full Access	✗ 403	✗ 403
Operating Hours Scheduling	✗ 401	✗ 403	✓ Full Access	✗ 403	✗ 403
Booking Accept / Reject / Complete	✗ 401	✗ 403	✓ Full Access	✗ 403	✗ 403
Direct Photo Multipart Upload	✗ 401	✗ 403	✓ Own Profile	✗ 403	✓ All Profiles
Support Ticket Queue (/support/**)	✗ 401	✗ 403	✗ 403	✓ Assigned Only	✓ Full Queue
Support Ticket Resolution	✗ 401	✗ 403	✗ 403	✓ Assigned Only	✓ Full Access
Admin Dashboard KPIs (/admin/**)	✗ 401	✗ 403	✗ 403	✗ 403	✓ Full Access
Provider Status Approval/Suspension	✗ 401	✗ 403	✗ 403	✗ 403	✓ Full Access
Global Ticket Re-assignment	✗ 401	✗ 403	✗ 403	✗ 403	✓ Full Access

Functional Area / Route / Endpoint	Public / Guest	CUSTOMER	PROVIDER	SUPPORT_AGENT	ADMIN
Help Center Article Authoring/CRUD	✗ 401	✗ 403	✗ 403	✗ 403	✓ Full Access
System Security Audit Logs	✗ 401	✗ 403	✗ 403	✗ 403	✓ Full Access

12. Customer Persona & End-to-End Workflow

The Customer workflow represents the demand side of the marketplace. Below is the verified operational sequence tested and confirmed in the application.

12.1 Customer Workflow Steps

- 1. Authentication:** Customer authenticates via `/customer/login` (`customer@serviceconnect.com` / `Password123!`). The client stores JWT tokens and redirects to `/customer/dashboard` .
- 2. Profile & Settings Inspection:** Navigating to `/customer/profile` and `/customer/settings` loads live profile data via `/api/users/me` and settings via `/api/users/me/settings` .
- 3. Catalog & Provider Discovery:**
 - Customer accesses `/customer/services` or `/services` to explore catalog offerings.
 - Filters offerings by category (`CLEANING` , `PLUMBING` , `ELECTRICAL`).
 - Selects a certified provider (e.g. Provider #7) to view their profile, portfolio photos, operating hours, and catalog services.
- 4. Service Request Creation:**
 - Customer selects a service (e.g. "Automation E2E Service 9490", \$150.00).
 - Chooses a scheduled date/time slot matching the provider's active hours.
 - Enters notes and submits the booking.
 - Frontend transmits `POST /api/v1/bookings/requests` with unique `X-Idempotency-Key` header. Booking is created with status `PENDING` .
- 5. Booking Tracking:** Customer views the booking on `/customer/bookings` and observes real-time status changes from `PENDING` → `ACCEPTED` → `COMPLETED` .
- 6. Payment Processing:** Accesses `/customer/payment/:bookingId` , reviews cost snapshot, enters simulated payment credentials, and confirms transaction.
- 7. Review Authoring:** Once the booking reaches `COMPLETED` , customer navigates to `/customer/reviews` , selects a 5-star rating, enters comments, and submits via `POST` .

/api/reviews .

8. **Support Ticketing:** If assistance is required, customer navigates to /customer/support , files Ticket #8 (category: TECHNICAL , priority: MEDIUM), and communicates with the support team via real-time threaded messaging.

13. Provider Persona & End-to-End Workflow

The Provider workflow represents the supply side of the marketplace.

13.1 Provider Workflow Steps

1. **Authentication:** Provider authenticates via /provider/login (provider@serviceconnect.com / Password123!), landing on /provider/dashboard .
2. **Onboarding & Business Profile:** Provider completes profile details at /provider/profile , configuring business name, bio, service radius, and address.
3. **Catalog Item Management:**
 - Provider accesses /provider/catalog .
 - Creates new service offerings via modal form (Title, Description, Category, Duration in minutes, Price in USD).
 - Executes POST /api/catalog . Offerings appear immediately in the provider's catalog and public marketplace.
4. **Operating Hours Scheduling:**
 - Provider navigates to /provider/availability .
 - Defines working shifts per day of the week (e.g. Monday–Friday 09:00 to 17:00).
 - Submits via POST /api/v1/providers/{id}/availability .
5. **Incoming Booking Request Lifecycle:**
 - Provider navigates to /provider/bookings .
 - Inspects incoming PENDING requests.
 - Executes **Accept:** Sends PATCH /api/v1/bookings/requests/{id}/status with { status: "ACCEPTED" } .
 - Executes **Complete:** Upon performing service, sends PATCH /api/v1/bookings/requests/{id}/status with { status: "COMPLETED" } .
6. **Portfolio Photos Direct Multipart Upload:**
 - Provider navigates to /provider/photos .
 - Chooses a local image file (PNG , JPG , or WEBP up to 5MB).

- Observes instant client-side thumbnail preview, file name, and formatted byte size.
- Clicks [Upload Photo]. Frontend executes `POST /api/v1/providers/{id}/photos/upload` as `multipart/form-data`.
- Photo is persisted to server disk and rendered in the gallery grid.
- Provider can delete photos at any time via `DELETE /api/v1/providers/{id}/photos/{photoId}`.

14. Support Agent Persona & End-to-End Workflow

The Support Agent is responsible for operational ticket triage, customer resolution, and customer success.

14.1 Support Agent Workflow Steps

- 1. Authentication & Routing:** Agent logs in at `/admin/login` using `agent@serviceconnect.com / Password123!`. `App.tsx` detects `ROLE_SUPPORT_AGENT` and routes directly to `/support/tickets`.
- 2. Assigned Ticket Queue:** Agent inspects tickets assigned to their account (`GET /api/v1/tickets/assigned`). The table displays ticket ID, customer ID, title, category badge, priority badge, and status.
- 3. Ticket Detail & Threaded Communication:**
 - Agent clicks a ticket (e.g. Ticket #8) to open `/support/tickets/8`.
 - Reads the customer's problem description.
 - Types an agent response and clicks `Send`. Frontend executes `POST /api/v1/tickets/8/agent/messages`.
 - Message renders in the threaded conversation with an `AGENT` sender badge.
- 4. Ticket Resolution:** Agent updates ticket status to `RESOLVED` via `PATCH /api/v1/tickets/8/assigned` with `{ status: "RESOLVED" }`.
- 5. Role Security Enforcement:** If the Support Agent attempts to navigate to `/admin/dashboard` or `/admin/audit-logs`, the system displays the `403 Forbidden` Access Denied screen.

15. Administrator Persona & End-to-End Workflow

The Administrator possesses supreme platform oversight and governance privileges.

15.1 Administrator Workflow Steps

1. **Admin Authentication:** Admin authenticates via `/admin/login` using `admin@serviceconnect.com` / `Password123!`, landing on `/admin/dashboard`.
2. **Platform KPI Metrics:** Reviews total active users, registered providers, completed bookings, gross platform volume, and pending support tickets.
3. **Provider Governance:**
 - Navigates to `/admin/providers`.
 - Filters providers by status (`PENDING_APPROVAL` , `APPROVED` , `SUSPENDED`).
 - Opens provider detail (`/admin/providers/7`), reviews business documents and photos.
 - Updates status via `PATCH /api/v1/providers/{id}/status` to approve or suspend accounts.
4. **Global Support Ticket Management:**
 - Navigates to `/admin/tickets`.
 - Inspects all platform tickets across all customers and agents.
 - Reassigns unassigned tickets to specific agents via `PATCH /api/v1/tickets/admin/{id}/assignment`.
5. **Help Center Content Authoring:**
 - Navigates to `/admin/help-center`.
 - Creates new articles at `/admin/help-center/new` specifying title, URL slug, Markdown content, category, and display order.
 - Articles instantly propagate to the public Help Center upon publishing.
6. **Security & Audit Log Inspection:**
 - Navigates to `/admin/audit-logs`.
 - Reviews system event logs, filtering by Actor ID, Role, Action Type, or Resource Type.

16. Help Center Domain System

The Help Center provides a self-service knowledge base for public visitors and authenticated users.

16.1 Architecture & Taxonomies

- **Categories:** Standardized enum taxonomy: `BOOKINGS` , `PAYMENTS` , `ACCOUNT` , `SERVICES` , `SAFETY` , `POLICIES` .

- **Category Slugs:** Handled via route `/help/category/:category` . Verified fix prevents `/undefined` by evaluating `typeof cat === 'string' ? cat : cat.name` .
- **Search Mechanism:** The backend executes pattern queries against published articles. Postgres query bug was resolved by introducing dedicated non-null search methods (`searchPublishedWithPattern`).
- **Article Viewer:** `/help/article/:slug` fetches article by unique slug, rendering sanitized HTML via `DOMPurify` .

17. Catalog & Service Discovery Domain System

The Catalog service manages the inventory of trade services published by providers.

17.1 Entity & Business Logic

- **Catalog Item Schema:** `id` , `provider_id` , `title` , `description` , `category` , `price` , `duration_minutes` , `active` , `created_at` , `updated_at` .
- **Public Discovery:** Unauthenticated guests can search catalog items via `GET /api/catalog` with optional `search` and `category` parameters.
- **Provider Isolation:** Providers can only modify or deactivate catalog items matching their authenticated `providerId` .

18. Booking Lifecycle & State Machine

Service bookings follow a strict, deterministic state machine enforced by the Booking Service.

18.1 State Machine Diagram

```
stateDiagram-v2
    [*] --> PENDING: Customer creates request (X-Idempotency-Key)

    PENDING --> ACCEPTED: Provider accepts request
    PENDING --> REJECTED: Provider rejects request
    PENDING --> CANCELLED: Customer cancels request

    ACCEPTED --> COMPLETED: Provider completes service
    ACCEPTED --> CANCELLED: Customer/Provider cancels

    REJECTED --> [*]
    CANCELLED --> [*]
    COMPLETED --> [*]: Unlocks Review submission
```

18.2 State Transition Rules & Validation

1. Creation:

- Customer submits requested slot (`scheduledStartTime` to `scheduledEndTime`).
- Service verifies slot falls within the provider's active `ProviderAvailability` windows.
- Captures immutable `priceSnapshot` from Catalog Item to prevent post-booking price changes.
- Enforces uniqueness via `X-Idempotency-Key` header.

2. Acceptance: Only the assigned provider can accept the booking, transitioning state to `ACCEPTED` .

3. Completion: Only the assigned provider can mark the booking as `COMPLETED` .

4. Cancellation: Customer can cancel while in `PENDING` or `ACCEPTED` status.

19. Review & Rating Domain System

The Review system provides trust and reputation metrics across the platform.

19.1 Verification Rules & Aggregation

- Eligibility:** A review can **only** be authored by the customer who initiated the booking, and only when the booking status is `COMPLETED` .
- Unique Constraint:** The database enforces a unique constraint on `booking_id` , guaranteeing that each booking receives at most one review.
- Rating Bounds:** Numeric integer between `1` and `5` stars.

- **Aggregate Recalculation:** Upon review creation or deactivation, the Review Service recalculates the provider's `ratingAverage` (double precision) and `ratingCount` (integer), updating the provider profile in real time.

20. Support Ticketing Domain System

The Support Ticketing domain facilitates structured resolution of customer inquiries and complaints.

20.1 Ticket Lifecycle & Prioritization

- **Categories:** `TECHNICAL` , `BILLING` , `GENERAL` , `ACCOUNT` , `SERVICE_QUALITY` .
- **Priorities:** `LOW` , `MEDIUM` , `HIGH` , `URGENT` .
- **Status Machine:** `OPEN` → `IN_PROGRESS` → `RESOLVED` → `CLOSED` .
- **Threading Model:** Each ticket links to an arbitrary number of `SupportMessage` entities. Messages record `senderId` and `senderType` (`CUSTOMER` , `AGENT` , `ADMIN`) to preserve a verifiable audit trail.

21. Provider Portfolio & Direct Photo Upload System

ServiceConnect features a fully functional direct photo upload architecture, replacing legacy URL input fields with binary multipart uploads.

21.1 End-to-End Upload Architecture

1. Frontend Interface (`/provider/photos`):

- Drag-and-drop / file selector input accepting `image/png` , `image/jpeg` , `image/webp` .
- Client-side validation enforcing 5MB maximum file size.
- Instant client-side thumbnail preview with formatted file size and filename.
- Single-click upload sending `FormData` via `POST` `/api/v1/providers/{providerId}/photos/upload` .

2. Axios Boundary Management:

- Axios request interceptor detects `FormData` and deletes the default `Content-Type: application/json` header, allowing browser to set `multipart/form-data; boundary=...` .

3. **Backend Processing (`ProviderPhotoController.java` & `ProviderPhotoService.java`):**

 - Validates MIME type and provider profile ownership.
 - Generates unique stored filename: `UUID.randomUUID() + "_" + originalFilename`.
 - Persists binary data to disk at `uploads/providers/{providerId}/{storedFilename}`.
 - Inserts `ProviderPhoto` record in `serviceconnect_provider` database.

4. **Public Image Serving:**
 - Public endpoint: `GET /api/v1/providers/{providerId}/photos/files/{filename}`
 - Returns `ResponseEntity<Resource>` with explicit `Content-Type` and `Cache-Control: public, max-age=604800`.
5. **Deletion:**
 - `DELETE /api/v1/providers/{providerId}/photos/{photoId}` removes the database record and deletes the physical file from server storage.

22. Database Schema & Data Models

ServiceConnect employs 8 dedicated PostgreSQL databases. Below are the verified relational schemas and column definitions.

22.1 `serviceconnect_auth`

- **users :**
 - `id` : BIGSERIAL PRIMARY KEY
 - `email` : VARCHAR(255) NOT NULL UNIQUE
 - `password_hash` : VARCHAR(255) NOT NULL
 - `role` : VARCHAR(50) NOT NULL (CUSTOMER, PROVIDER, SUPPORT_AGENT, ADMIN)
 - `active` : BOOLEAN DEFAULT TRUE
 - `email_verified` : BOOLEAN DEFAULT FALSE
 - `phone_verified` : BOOLEAN DEFAULT FALSE
 - `created_at` : TIMESTAMP WITH TIME ZONE DEFAULT NOW()
 - `updated_at` : TIMESTAMP WITH TIME ZONE DEFAULT NOW()
- **refresh_tokens :**
 - `id` : BIGSERIAL PRIMARY KEY
 - `user_id` : BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE
 - `token_hash` : VARCHAR(255) NOT NULL UNIQUE
 - `expires_at` : TIMESTAMP WITH TIME ZONE NOT NULL
 - `revoked` : BOOLEAN DEFAULT FALSE
 - `created_at` : TIMESTAMP WITH TIME ZONE DEFAULT NOW()

- **verification_tokens & verification_challenges :**
 - `id` : BIGSERIAL PRIMARY KEY
 - `user_id` : BIGINT REFERENCES users(id)
 - `token` : VARCHAR(255) NOT NULL UNIQUE
 - `code_hash` : VARCHAR(255)
 - `channel` : VARCHAR(50) (EMAIL, PHONE)
 - `purpose` : VARCHAR(50)
 - `expires_at` : TIMESTAMP WITH TIME ZONE NOT NULL

22.2 serviceconnect_user

- **user_profiles :**
 - `id` : BIGINT PRIMARY KEY (Matches auth user ID)
 - `first_name` : VARCHAR(100) NOT NULL
 - `last_name` : VARCHAR(100) NOT NULL
 - `phone_number` : VARCHAR(50)
 - `avatar_url` : VARCHAR(500)
 - `address` : VARCHAR(255)
 - `city` : VARCHAR(100)
 - `state` : VARCHAR(100)
 - `postal_code` : VARCHAR(20)
 - `bio` : TEXT
 - `created_at` : TIMESTAMP WITH TIME ZONE DEFAULT NOW()
 - `updated_at` : TIMESTAMP WITH TIME ZONE DEFAULT NOW()

22.3 serviceconnect_provider

- **providers :**
 - `id` : BIGSERIAL PRIMARY KEY
 - `user_id` : BIGINT NOT NULL UNIQUE
 - `business_name` : VARCHAR(255) NOT NULL
 - `bio` : TEXT
 - `address` : VARCHAR(255)
 - `city` : VARCHAR(100)
 - `state` : VARCHAR(100)
 - `postal_code` : VARCHAR(20)

- latitude : NUMERIC(10, 8)
- longitude : NUMERIC(11, 8)
- status : VARCHAR(50) DEFAULT 'PENDING_APPROVAL' (APPROVED, SUSPENDED)
- rating_average : NUMERIC(3, 2) DEFAULT 0.00
- rating_count : INTEGER DEFAULT 0
- created_at : TIMESTAMP WITH TIME ZONE DEFAULT NOW()
- updated_at : TIMESTAMP WITH TIME ZONE DEFAULT NOW()
- **provider_availabilitys :**
 - id : BIGSERIAL PRIMARY KEY
 - provider_id : BIGINT NOT NULL REFERENCES providers(id) ON DELETE CASCADE
 - day_of_week : INTEGER NOT NULL (1 = Monday, 7 = Sunday)
 - start_time : TIME NOT NULL
 - end_time : TIME NOT NULL
 - active : BOOLEAN DEFAULT TRUE
- **provider_photos :**
 - id : BIGSERIAL PRIMARY KEY
 - provider_id : BIGINT NOT NULL REFERENCES providers(id) ON DELETE CASCADE
 - image_url : VARCHAR(500) NOT NULL
 - display_order : INTEGER DEFAULT 0
 - created_at : TIMESTAMP WITH TIME ZONE DEFAULT NOW()

22.4 serviceconnect_catalog

- **catalog_items :**
 - id : BIGSERIAL PRIMARY KEY
 - provider_id : BIGINT NOT NULL
 - title : VARCHAR(255) NOT NULL
 - description : TEXT
 - category : VARCHAR(100) NOT NULL
 - price : NUMERIC(10, 2) NOT NULL
 - duration_minutes : INTEGER NOT NULL
 - active : BOOLEAN DEFAULT TRUE
 - created_at : TIMESTAMP WITH TIME ZONE DEFAULT NOW()
 - updated_at : TIMESTAMP WITH TIME ZONE DEFAULT NOW()

22.5 serviceconnect_booking

- **service_requests :**
 - **id :** BIGSERIAL PRIMARY KEY
 - **customer_id :** BIGINT NOT NULL
 - **provider_id :** BIGINT NOT NULL
 - **catalog_item_id :** BIGINT NOT NULL
 - **scheduled_start_time :** TIMESTAMP WITH TIME ZONE NOT NULL
 - **scheduled_end_time :** TIMESTAMP WITH TIME ZONE NOT NULL
 - **price_snapshot :** NUMERIC(10, 2) NOT NULL
 - **currency :** VARCHAR(10) DEFAULT 'USD'
 - **status :** VARCHAR(50) NOT NULL (PENDING, ACCEPTED, REJECTED, COMPLETED, CANCELLED)
 - **notes :** TEXT
 - **cancellation_reason :** VARCHAR(255)
 - **created_at :** TIMESTAMP WITH TIME ZONE DEFAULT NOW()
 - **updated_at :** TIMESTAMP WITH TIME ZONE DEFAULT NOW()

22.6 serviceconnect_review

- **reviews :**
 - **id :** BIGSERIAL PRIMARY KEY
 - **booking_id :** BIGINT NOT NULL UNIQUE
 - **customer_id :** BIGINT NOT NULL
 - **provider_id :** BIGINT NOT NULL
 - **rating :** INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5)
 - **comment :** TEXT
 - **active :** BOOLEAN DEFAULT TRUE
 - **created_at :** TIMESTAMP WITH TIME ZONE DEFAULT NOW()
 - **updated_at :** TIMESTAMP WITH TIME ZONE DEFAULT NOW()

22.7 serviceconnect_payment

- **payments :**
 - **id :** UUID PRIMARY KEY
 - **booking_id :** UUID NOT NULL
 - **user_id :** UUID NOT NULL

- `amount` : `NUMERIC(10, 2) NOT NULL`
- `currency` : `VARCHAR(10) DEFAULT 'USD'`
- `status` : `VARCHAR(50) NOT NULL` (PENDING, COMPLETED, FAILED)
- `payment_method` : `VARCHAR(50)`
- `transaction_ref` : `VARCHAR(255) UNIQUE`
- `created_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`
- `updated_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`

22.8 serviceconnect_admin

- **support_tickets :**
 - `id` : `BIGSERIAL PRIMARY KEY`
 - `customer_id` : `BIGINT NOT NULL`
 - `assigned_agent_id` : `BIGINT`
 - `title` : `VARCHAR(255) NOT NULL`
 - `description` : `TEXT NOT NULL`
 - `category` : `VARCHAR(50) NOT NULL`
 - `priority` : `VARCHAR(50) NOT NULL`
 - `status` : `VARCHAR(50) NOT NULL` (OPEN, IN_PROGRESS, RESOLVED, CLOSED)
 - `created_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`
 - `updated_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`
- **support_messages :**
 - `id` : `BIGSERIAL PRIMARY KEY`
 - `ticket_id` : `BIGINT NOT NULL REFERENCES support_tickets(id) ON DELETE CASCADE`
 - `sender_id` : `BIGINT NOT NULL`
 - `sender_type` : `VARCHAR(50) NOT NULL` (CUSTOMER, AGENT, ADMIN)
 - `content` : `TEXT NOT NULL`
 - `created_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`
- **help_articles :**
 - `id` : `BIGSERIAL PRIMARY KEY`
 - `slug` : `VARCHAR(255) NOT NULL UNIQUE`
 - `title` : `VARCHAR(255) NOT NULL`
 - `content` : `TEXT NOT NULL`
 - `category` : `VARCHAR(100) NOT NULL`
 - `display_order` : `INTEGER DEFAULT 0`
 - `published` : `BOOLEAN DEFAULT TRUE`

- `created_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`
 - `updated_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`
 - **audit_logs** :
 - `id` : `BIGSERIAL PRIMARY KEY`
 - `actor_id` : `BIGINT NOT NULL`
 - `actor_role` : `VARCHAR(50) NOT NULL`
 - `action` : `VARCHAR(100) NOT NULL`
 - `resource_type` : `VARCHAR(100) NOT NULL`
 - `resource_id` : `VARCHAR(100)`
 - `details` : `TEXT`
 - `ip_address` : `VARCHAR(100)`
 - `created_at` : `TIMESTAMP WITH TIME ZONE DEFAULT NOW()`
-

23. Inter-Service Communication & Service Discovery

ServiceConnect utilizes Spring Cloud Netflix Eureka for service registry and client-side discovery.

23.1 Registration & Discovery Mechanics

- **Eureka Server:** Runs on `http://localhost:8761/eureka/`.
 - **Client Registration:** Microservices declare `eureka.client.service-url.defaultZone` and `eureka.instance.prefer-ip-address = true`.
 - **OpenFeign Declarative Clients:**
 - `BookingService` communicates with `ProviderServiceClient` to resolve provider status and operating hours.
 - `AdminService` communicates with `ProviderServiceClient` to update provider verification status.
 - `ReviewService` communicates with `BookingServiceClient` to verify that booking status is `COMPLETED` prior to persisting reviews.
 - **Resilience:** Feign clients are configured with connect timeouts of 3000ms and read timeouts of 5000ms.
-

24. API Gateway Architecture & Routing Table

The Spring Cloud Gateway (`api-gateway`) on port `8080`) functions as the system's edge controller.

24.1 Routing Table Configuration

```

routes:
- id: auth-service
  uri: lb://AUTH-SERVICE
  predicates:
    - Path=/api/v1/auth/**
  filters:
    - name: CircuitBreaker
      args:
        name: authCircuitBreaker
        fallbackUri: forward:/fallback/service

- id: user-service
  uri: lb://USER-SERVICE
  predicates:
    - Path=/api/v1/users/**
  filters:
    - name: CircuitBreaker
      args:
        name: userCircuitBreaker
        fallbackUri: forward:/fallback/service

- id: provider-service
  uri: lb://PROVIDER-SERVICE
  predicates:
    - Path=/api/v1/providers/**
  filters:
    - name: CircuitBreaker
      args:
        name: providerCircuitBreaker
        fallbackUri: forward:/fallback/service

- id: booking-service
  uri: lb://BOOKING-SERVICE
  predicates:
    - Path=/api/v1/bookings/**
  filters:
    - name: CircuitBreaker
      args:
        name: bookingCircuitBreaker
        fallbackUri: forward:/fallback/service

- id: catalog-service
  uri: lb://CATALOG-SERVICE
  predicates:
    - Path=/api/v1/catalog/**
  filters:
    - name: CircuitBreaker
      args:
        name: catalogCircuitBreaker
        fallbackUri: forward:/fallback/service

- id: review-service
  uri: lb://REVIEW-SERVICE
  predicates:
    - Path=/api/v1/reviews/**

```

```

filters:
  - RewritePath=/api/v1/reviews/(?<segment>.*), /api/reviews/${segment\}
  - name: CircuitBreaker
    args:
      name: reviewCircuitBreaker
      fallbackUri: forward:/fallback/service

- id: help-center
  uri: lb://ADMIN-SERVICE
  predicates:
    - Path=/api/v1/help-center/**
  filters:
    - name: CircuitBreaker
      args:
        name: adminCircuitBreaker
        fallbackUri: forward:/fallback/service

```

24.2 Rate Limiting & Circuit Breaking

- **Redis Rate Limiter:** Configured on authentication routes (`/login` , `/register` , `/password/forgot`) with IP-based key resolving (`ipKeyResolver`).
- **Resilience4j:** Configured with sliding count windows of 10 requests, 50% failure rate threshold, and 10-second wait in open state before half-open probing.

25. Security Architecture & Threat Model

25.1 Security Controls & Policies

- **Authentication:** Stateless HMAC-SHA256 JWT tokens with 15-minute lifespan.
- **Password Hashing:** BCrypt password encoder with 12 salt rounds in `auth-service` .
- **CORS Architecture:** Configured across Gateway and microservices. For local development, the Vite dev proxy bridges origins transparently.
- **SQL Injection Defense:** 100% parameterized queries via Spring Data JPA and Hibernate ORM.
- **Input Validation:** Jakarta Bean Validation annotations (`@Valid` , `@NotNull` , `@Positive` , `@Min` , `@Max`) on all controller request bodies.
- **XSS Sanitization:** React automatically escapes text nodes. Markdown renderings in Help Center articles are explicitly passed through `DOMPurify.sanitize()` .

26. Error Handling & Exception Management

26.1 Backend Global Exception Handler

Each Spring Boot service implements a `@RestControllerAdvice` mapping uncaught exceptions into standard JSON error objects:

```
{
  "timestamp": "2026-09-19T08:52:04.123Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Provider operating hours do not cover the requested time slot",
  "path": "/api/v1/bookings/requests"
}
```

26.2 Frontend Error Handling

- **Toast Feedback:** Sonner displays context-aware success and error notifications.
- **React ErrorBoundary:** Top-level component catches rendering errors and displays a user-friendly crash screen with recovery action.
- **Axios 401 Handling:** Interceptor automatically attempts token refresh; redirects to `/session-expired` only upon total authentication failure.

27. Configuration & Environment Variables

The table below catalogs the environment variables and default configuration keys used across the ecosystem.

Variable Name	Service Scope	Default / Example	Purpose / Description	Sensitive?
<code>DB_URL</code>	All microservices	<code>jdbc:postgresql://localhost:5432/<db_name></code>	PostgreSQL JDBC connection URL	No
<code>DB_USERNAME</code>	All microservices	<code>postgres</code>	PostgreSQL database user	No
<code>DB_PASSWORD</code>	All microservices	<code><REDACTED></code>	PostgreSQL user password	YES

Variable Name	Service Scope	Default / Example	Purpose / Description	Sensitive?
JWT_SECRET	Auth & Gateway	<REDACTED> (Min 256 bits)	Secret key for signing and validating JWTs	YES
JWT_ISSUER	Auth & Gateway	serviceconnect-auth	JWT token issuer claim	No
REDIS_HOST	api-gateway	localhost	Redis server hostname for rate limiting	No
REDIS_PORT	api-gateway	6379	Redis server port	No
EUREKA_SERVER_URL	All microservices	http://localhost:8761/eureka/	Service discovery default zone URL	No
VITE_API_GATEWAY_URL	frontend	http://localhost:8080	Base gateway URL in production	No
VITE_ADMIN_SERVICE_URL	frontend	http://localhost:8083	Direct Admin Service URL	No

28. Comprehensive Test Evidence & Verification History

A milestone-level End-to-End automated verification suite was executed using visible Google Chrome (C:\Program Files\Google\Chrome\Application\chrome.exe).

28.1 Test Results Summary

- Overall Suite Status: 100% PASSING (18 / 18 Phases Passed)
- HTTP Network Failures: 0
- Unhandled Console Errors: 0
- Captured Artifacts: 32 Verified Screenshots

28.2 Verified Screenshot Inventory

Artifact Filename	Phase / Flow	Verification Details
00_public_landing.png	Phase 1	Landing page renders hero, categories, features, responsive nav
00_public_services.png	Phase 1	Public services catalog renders 13 verified items without auth
00_public_providers.png	Phase 1	Public provider directory renders 5 providers and map markers
00_public_help.png	Phase 1	Public Help Center renders categories and search without HTTP 500
01_customer_dashboard.png	Phase 2	Customer logs in and accesses dashboard summary cards
03_provider_service_created.png	Phase 3	Provider publishes "Automation E2E Service 9490" (\$150)
04_customer_sees_service.png	Phase 4	Customer views the newly published service in catalog
05_customer_creates_request.png	Phase 4	Customer books service slot; Booking #14 created (PENDING)
06_provider_sees_request.png	Phase 5	Provider views Booking #14 in incoming requests queue
07_provider_accepts_request.png	Phase 5	Provider accepts booking; status updates to ACCEPTED
08_customer_sees_accepted_status.png	Phase 6	Customer portal displays ACCEPTED status badge
09_provider_completes_service.png	Phase 7	Provider marks service as COMPLETED
10_customer_sees_completed_status.png	Phase 8	Customer portal displays COMPLETED status
11_review_submitted.png	Phase 9	Customer posts 5-star review with comments for Booking #14
12_customer_creates_ticket.png	Phase 10	Customer opens Ticket #8 (TECHNICAL , MEDIUM)
13_support_agent_opens_ticket.png	Phase 11	Support agent views Ticket #8 in assigned queue
14_support_agent_resolves_ticket.png	Phase 11	Agent posts response message and marks ticket RESOLVED
15_customer_sees_final_ticket_status.png	Phase 12	Customer confirms Ticket #8 marked Resolved
16_admin_dashboard.png	Phase 13	Admin accesses metrics dashboard with platform statistics
17_admin_ticket_management.png	Phase 13	Admin manages tickets, filters by status, reassigns
18_admin_help_center.png	Phase 13	Admin Help Center management table and article editor

Artifact Filename	Phase / Flow	Verification Details
19_admin_audit_logs.png	Phase 13	Admin reviews audit logs with actor and action filters
20_role_denial_screen.png	Phase 14	Customer attempting to open <code>/admin/dashboard</code> gets 403 Forbidden
21_fixed_help_category.png	Phase 0	Help category navigation verified at <code>/help/category/BOOKINGS</code>
22_fixed_services_page.png	Phase 0	Catalog renders 13 public services with unauthenticated access
23_fixed_providers_page.png	Phase 0	Provider directory renders 5 active providers
24_fixed_provider_detail.png	Phase 0	Provider #7 public profile with operating hours and photos
25_customer_profile_verified.png	Follow-up	Customer profile renders live profile data via proxy
26_customer_settings_verified.png	Follow-up	Customer settings renders live account preferences
27_provider_photos_empty_state.png	Photo Feature	Provider photo page displays upload box and clean empty state
28_provider_photo_selected_preview.png	Photo Feature	Selected image shows client preview, filename, and file size
29_provider_photo_uploaded_grid.png	Photo Feature	Uploaded photo appears in provider portfolio gallery
30_provider_photo_persisted_after_refresh.png	Photo Feature	Photo persists after hard browser refresh
31_customer_sees_provider_portfolio_photo.png	Photo Feature	Public/Customer view confirms portfolio photo rendering
32_provider_photo_deleted_clean_state.png	Photo Feature	Photo deleted; database record and disk file cleanly removed

29. Bug Fix History & Resolution Log

During milestone integration testing, 7 targeted bugs were identified, debugged, and permanently resolved.

29.1 Fix Log

1. Issue A — Public Services Catalog HTTP 401:

- *Root Cause:* `CatalogItemController.java` declared `@RequestHeader HttpHeaders.AUTHORIZATION` as mandatory, throwing HTTP 401 for unauthenticated visitors.

- *Resolution:* Made `@RequestHeader(value = AUTHORIZATION, required = false)` optional and added `permitAll()` on `GET /api/catalog/**` in `catalog-service SecurityConfig.java`.

2. Issue B — Public Providers Directory HTTP 401:

- *Root Cause:* Gateway and `provider-service` security rules restricted `/api/v1/providers/**` strictly to `ROLE_PROVIDER` and `ROLE_ADMIN`.
- *Resolution:* Updated `provider-service SecurityConfig.java` to permit unauthenticated `GET` requests on `/api/v1/providers`, `/api/v1/providers/{id}/public`, `/api/v1/providers/{id}/availability/active`, and `photos`. Updated Axios response interceptor to prevent redirecting guests to `/session-expired`.

3. Issue C — Help Center Articles HTTP 500:

- *Root Cause:* Postgres threw syntax/runtime error when a null parameter was supplied to `LIKE LOWER(CONCAT('%', :search, '%'))`.
- *Resolution:* Created dedicated Spring Data methods `findByPublishedTrueOrderByDisplayOrderAscTitleAsc` (for null search) and `searchPublishedWithPattern` (for active queries).

4. Issue D — Help Center Category /undefined :

- *Root Cause:* Frontend link builder in `HelpCenter.tsx` assumed category items were objects with a `.name` property, but the backend returned raw strings `["BOOKINGS"]`.
- *Resolution:* Updated link generation to `typeof cat === 'string' ? cat : cat.name`, correctly targeting `/help/category/BOOKINGS`.

5. Issue E — Review Gateway Route 404:

- *Root Cause:* Gateway routed `/api/v1/reviews/**` directly without stripping the `/v1` segment, but `review-service` listened on `/api/reviews/**`.
- *Resolution:* Added `RewritePath=/api/v1/reviews/(?<segment>.*), /api/reviews/${segment\}` in `api-gateway application.yml`.

6. Issue F — Direct Port 8082 CORS Preflight Failure:

- *Root Cause:* Direct browser requests to `http://localhost:8082/api/users/me` failed browser CORS preflight checks because User Service did not expose permissive CORS headers.
- *Resolution:* Configured Vite dev proxy in `vite.config.ts` to route `/api/users` to `http://localhost:8082`, and set `USER_SERVICE_URL = ''` in `src/lib/axios.ts`.

7. Issue G — Multipart Photo Upload Boundary Missing:

- *Root Cause:* Axios default header `'Content-Type': 'application/json'` overrode browser multipart boundaries when transmitting `FormData`.

- *Resolution:* Added condition in Axios request interceptor: `if (config.data instanceof FormData) delete config.headers['Content-Type']`.

30. Implementation Status Matrix

The matrix below reflects the actual implementation state of all platform features.

Module / Feature	Status	Notes
User Registration & Login	FULLY IMPLEMENTED	BCrypt encryption, JWT issuance, refresh tokens
User Profile & Account Settings	FULLY IMPLEMENTED	Profiles, phone, address, communication settings
Email & Phone Verification Codes	FULLY IMPLEMENTED	Token challenge/response endpoints in Auth Service
Provider Discovery & Public Profiles	FULLY IMPLEMENTED	Public directory, Leaflet map markers, bios
Provider Catalog Management	FULLY IMPLEMENTED	Create, edit, activate, deactivate services
Provider Operating Hours	FULLY IMPLEMENTED	Day of week, start/end shift scheduling
Direct Multipart Portfolio Photos	FULLY IMPLEMENTED	Direct file upload (PNG/JPG/WEBP), disk persistence, deletion
Booking Creation & Idempotency	FULLY IMPLEMENTED	Idempotency keys, slot validation, price snapshot
Booking Status Workflow	FULLY IMPLEMENTED	PENDING → ACCEPTED → COMPLETED lifecycle
Simulated Card Checkout	FULLY IMPLEMENTED	Payment form, transaction verification
Post-Service Reviews & Ratings	FULLY IMPLEMENTED	1-5 star ratings, single-review constraint, aggregate scores
Customer Support Ticketing	FULLY IMPLEMENTED	Category, priority, bidirectional threaded messaging
Support Agent Portal	FULLY IMPLEMENTED	Assigned ticket queue, agent messages, ticket resolution
Admin Platform KPI Dashboard	FULLY IMPLEMENTED	High-level business metrics and system counts
Admin Provider Governance	FULLY IMPLEMENTED	Approval, suspension, document review
Admin Help Center Manager	FULLY IMPLEMENTED	Article CRUD, publishing, slug management
Security Audit Logs	FULLY IMPLEMENTED	System event logging, actor/role/action filters
Role-Based Route Protection	FULLY IMPLEMENTED	ProtectedRoute wrapper, 403 Forbidden screen
Real-time WebSocket Notifications	NOT IMPLEMENTED / PLANNED	Currently utilizes TanStack Query polling and re-fetch
Stripe Production Webhook Ingestion	NOT IMPLEMENTED / PLANNED	Simulated payment controller implemented for testability

Module / Feature	Status	Notes
Distributed Async Message Bus	NOT IMPLEMENTED / PLANNED	Inter-service calls currently synchronous REST via Feign

31. Known Gaps, Limitations & Technical Debt

- 1. Synchronous Inter-Service Calls:** Microservices interact via synchronous OpenFeign REST clients rather than an asynchronous event bus (e.g. Apache Kafka or RabbitMQ). Network latency or temporary downstream unavailability can cascade if circuit breakers trip.
- 2. Local Disk Asset Storage:** Provider portfolio photos are currently stored on the local filesystem under `uploads/providers/{providerId}/`. In a distributed multi-instance deployment, this requires shared network volumes (NFS) or migration to cloud object storage (AWS S3 / Google Cloud Storage).
- 3. Payment Service UUID Identifier Mismatch:** The Payment Service uses `UUID` primary keys for `bookingId` and `userId`, whereas Auth and Booking services utilize `Long` identifiers. This is bridged in the frontend client adapter (`src/api/payment.ts`).
- 4. Vite Development Proxy Reliance:** Local frontend development relies on `vite.config.ts` proxy rules to bypass CORS on direct microservice ports. In staging and production, all traffic should route strictly through the API Gateway or Kubernetes Ingress.

32. Local Development & Setup Guide

32.1 Prerequisites

- **Java Development Kit (JDK):** OpenJDK 17 or higher
- **Build Tool:** Apache Maven 3.9+
- **Node.js:** Node 20+ and npm 10+
- **Database:** PostgreSQL 15+ running on port `5432`
- **Cache:** Redis 7+ running on port `6379`

32.2 Database Initialization

Execute the following PostgreSQL script to initialize the 8 microservice databases:

```
CREATE DATABASE serviceconnect_auth;
CREATE DATABASE serviceconnect_user;
CREATE DATABASE serviceconnect_admin;
CREATE DATABASE serviceconnect_provider;
CREATE DATABASE serviceconnect_booking;
CREATE DATABASE serviceconnect_catalog;
CREATE DATABASE serviceconnect_review;
CREATE DATABASE serviceconnect_payment;
```

32.3 Startup Sequence

Start services in the following order to ensure discovery and dependencies resolve cleanly:

1. Service Discovery (Port 8761):

```
cd Back-End-Services/service-discovery
mvn spring-boot:run
```

2. API Gateway (Port 8080):

```
cd Back-End-Services/api-gateway
mvn spring-boot:run
```

3. Domain Microservices (Ports 8081–8088): Launch `auth-service` , `user-service` , `admin-service` , `provider-service` , `booking-service` , `catalog-service` , `review-service` , and `payment-service` .

4. Frontend Single-Page App (Port 3000):

```
cd cool-bell
npm install
npm run dev
```

32.4 Verified Test Credentials

Role	Email	Password	User ID	Default Landing Page
Customer	customer@serviceconnect.com	Password123!	22	/customer/dashboard
Provider	provider@serviceconnect.com	Password123!	23 (Provider #7)	/provider/dashboard
Admin	admin@serviceconnect.com	Password123!	24	/admin/dashboard
Support Agent	agent@serviceconnect.com	Password123!	25	/support/tickets

33. Production Readiness & Deployment Considerations

33.1 Containerization (Docker)

Each microservice should be packaged using a multi-stage Dockerfile:

```
# Example for Spring Boot Microservice
FROM eclipse-temurin:17-jdk-jammy AS builder
WORKDIR /workspace
COPY . .
RUN ./mvnw clean package -DskipTests

FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --from=builder /workspace/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

33.2 Kubernetes Orchestration

- **Deployments & Services:** Deploy each microservice as a Kubernetes `Deployment` with a minimum of 2 replicas, fronted by a `ClusterIP` Service.
- **ConfigMaps & Secrets:** Externalize database credentials, Redis connections, and `JWT_SECRET` into Kubernetes `Secret` objects.
- **Ingress Controller:** Configure an NGINX Ingress Controller terminating TLS and routing `/api/**` to the `api-gateway` and `/**` to the compiled frontend static container.

33.3 Cloud Object Storage Migration

To support horizontal scaling of `provider-service`, replace local disk storage with Amazon S3 or Google Cloud Storage. The service generates presigned URLs for client uploads and serves public portfolio images via AWS CloudFront CDN.

34. Complete Repository Directory Tree

34.1 Frontend Repository Tree (cool-bell)

```

cool-bell/
├── dist/                # Production build output
├── public/              # Static media and favicons
├── src/
│   ├── api/            # 10 Domain API clients
│   ├── components/     # Reusable UI, auth guards, dialogs
│   ├── hooks/          # Custom React hooks
│   ├── layouts/        # 5 Structural layouts
│   ├── lib/            # Axios clients & Tailwind utilities
│   ├── pages/
│   │   ├── admin/      # 11 Admin pages
│   │   ├── customer/   # 22 Customer pages
│   │   ├── provider/   # 14 Provider pages
│   │   ├── public/     # 14 Public & error pages
│   │   └── support/    # 2 Support agent pages
│   ├── store/          # Zustand persistent auth store
│   ├── types/          # Centralized TypeScript schemas
│   ├── App.tsx         # Route hierarchy & role guards
│   ├── main.tsx        # React application bootstrap
│   └── index.css       # Global Tailwind CSS
├── package.json        # npm dependencies
├── tailwind.config.js  # Tailwind theme configurations
├── tsconfig.json       # TypeScript compiler config
└── vite.config.ts      # Vite server & proxy rules

```

34.2 Backend Services Repository Tree (Back-End-Services)

```

Back-End-Services/
├── admin-service/      # Port 8083 (DB: serviceconnect_admin)
├── api-gateway/        # Port 8080 (Redis, Resilience4j)
├── auth-service/       # Port 8081 (DB: serviceconnect_auth)
├── booking-service/    # Port 8085 (DB: serviceconnect_booking)
├── catalog-service/    # Port 8086 (DB: serviceconnect_catalog)
├── payment-service/    # Port 8088 (DB: serviceconnect_payment)
├── provider-service/   # Port 8084 (DB: serviceconnect_provider)
├── review-service/     # Port 8087 (DB: serviceconnect_review)
├── service-discovery/  # Port 8761 (Eureka Server)
└── user-service/       # Port 8082 (DB: serviceconnect_user)

```

35. Final Summary & Technical Sign-off

35.1 Technical Validation Statement

This technical documentation represents a comprehensive, source-verified audit of the **ServiceConnect** application. All architecture diagrams, port numbers, endpoint contracts, database schemas, and workflows have been directly confirmed against active source code and validated via a 100% passing End-to-End automated milestone test suite.

35.2 System Sign-off Block

- **Platform Health:** Operational & Passing
- **Verified Microservices:** 10 / 10 Active
- **Frontend Routes:** 63 Registered & Protected
- **Test Artifacts:** 32 Verified Screenshot Evidences
- **Lead Architecture Sign-off:** Antigravity Autonomous Pair Programmer
- **Date of Verification:** September 19, 2026